Lesson 9 Image Processing - Smoothing

1. Image Noise

During collecting, processing and transferring, the digital image will be disturbed by different noises, which leads to low-quality image, obscure image and disappeared image feature. And image smoothing is to improve the image by removing noise, and salt-and-pepper noise as well as Gauss noise are common.

1.1 Salt-and-pepper Noise

Salt-and-pepper noise is also known as pulse noise which is white dots and black dots appearing randomly, like there are black pixels in bright area and white pixels in dark area.

At left is the original picture and the right is the picture with salt-and-pepper noise.

1.2 Gauss Noise

Gauss noise is a term from signal processing theory denoting a kind of signal noise that has a probability density function (pdf) equal to that of the normal distribution (which is also known as the Gaussian distribution). Commonly, it can be suppressed by mathematical statistics.

At left is the original picture and the right is the picture with Gauss noise.

2. Image Smoothing

From the perspective of signal, image smoothing is to filter the high frequency part of the signal and reserve the low frequency part.

Based on filter, filtering can be divided into mean filtering, Gaussian filtering and median filtering.

2.1 Mean Filtering

The idea of mean filtering is simply to take the mean of all the pixels of the image that is assign the mean of all the pixels in the unit of a square to the center pixel.

Take the picture below as example. In picture (a), the center pixel value is

"**226**" and the mean of all the pixels is "**122**" obtained from the equation below, and "**122**" is the new center pixel value.

$$(40 + 107 + 5 + 198 + 226 + 223 + 37 + 68 + 193) \div 9 = 122$$

Replace the original center pixel value by "122" as the picture (b) shown.

40	107	5
198	226	223
37	68	193

(a)

40	107	5
198	122	223
37	68	193

(b)

The algorithm of the mean filtering is simple and its calculation speed is fast. However, the details of the image are destroyed during removing noise resulting in low definition.

2.2 Gauss Filtering

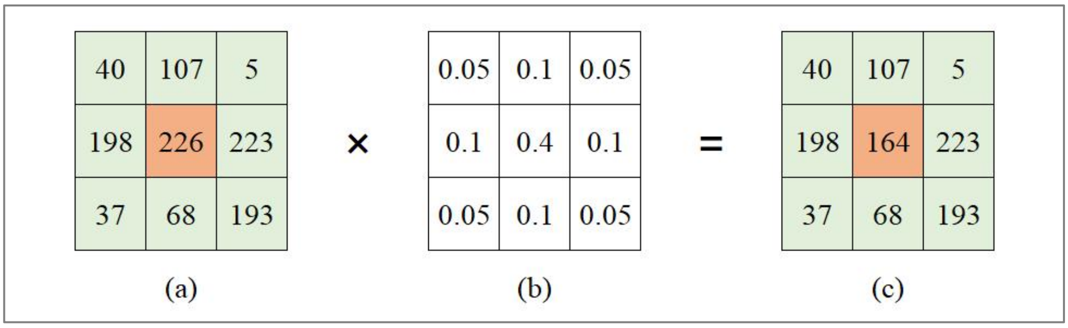
The weighted mean is calculated by multiplying each value by the corresponding weight, adding the sum, and then dividing the sum by the number of the values.

Gauss filtering is to obtain the weighted mean of all the pixels of the image that is assign the weighted mean in the unit of a square to the center pixel.

Take the picture below as example. In picture (a), the center pixel value is "**226**" and the weighted mean of all the pixels is "**164**" obtained from the equation below, and "**164**" is the new center pixel value.

$$40 \times 0.05 + 107 \times 0.1 + 5 \times 0.05 + 198 \times 0.1 + 226 \times 0.4 + 223 \times 0.1 + 37 \times 0.05 + 68 \times 0.1 + 193 \times 0.05 = 164$$

Replace the original center pixel value by “122” as the picture (c) shown.



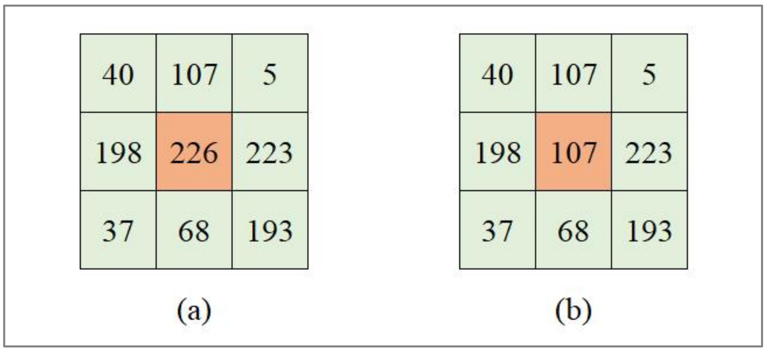
2.3 Median Filtering

The median is the middle value when a data set is ordered from least to greatest.

Median filtering is to take the median of all the pixels of the image that is assign the median in the unit of square to the center pixel.

Take the picture below as example. In picture (a), the center pixel value is “226” and the medium of the pixels is “**107**”, and “**107**” is the new center pixel value.

Replace the original center pixel value by “107” as the picture (b) shown.



3.Operation Steps

This routine will execute mean filtering, Gauss filtering and median filtering separately.

Before operation, please copy the routine “**filtering.py**” in “4.OpenCV->Lesson 9 Image Processing --- Smoothing->Routine Code” to the shared folder.

For how to configure the shared folder, please refer to the file in “**2. Linux Basic Lesson->Lesson 3 Linux Installation and Source Replacement**”.

Note: the input command should be case sensitive and the keywords can be complemented by “Tab” key.

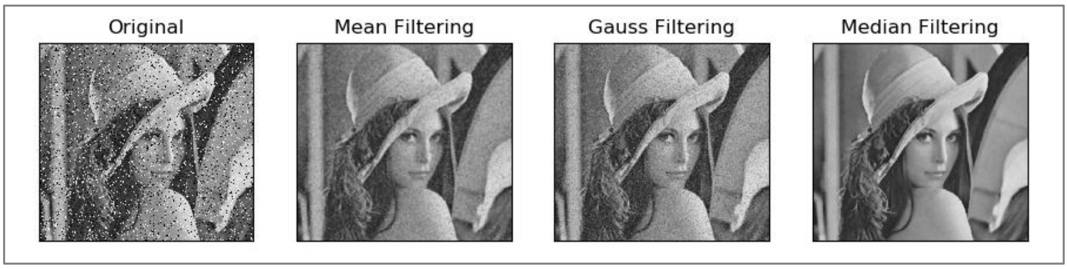
1） Open virtual machine and start the system. Click “”, and then “” or press “**Ctrl+Alt+T**” to open command line terminal.

2） Input command “**cd /mnt/hgfs/Share/**” and press Enter to enter the shared folder.

3） Input command “**python3 filtering.py**” and press Enter to run the routine.

4.Program Outcome

The final output image is as follow.



5.Program Analysis

The routine “**filtering.py**” can be found in “4.OpenCV->Lesson 9 Image Processing --- Smoothing->Routine Code”.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# image reading
img = cv2.imread('noise.jpg')

# image smoothing
blur1 = cv2.blur(img, (5, 5))    # mean filtering
blur2 = cv2.GaussianBlur(img, (5, 5), 1)     # Gauss filtering
blur3 = cv2.medianBlur(img, 5)  # Median filtering

# image display
plt.figure(figsize=(10, 5), dpi=100)
plt.rcParams['axes.unicode_minus'] = False
plt.subplot(141), plt.imshow(img), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(142), plt.imshow(blur1), plt.title("Mean Filtering")
plt.xticks([]), plt.yticks([])
plt.subplot(143), plt.imshow(blur2), plt.title("Gauss Filtering")
plt.xticks([]), plt.yticks([])
plt.subplot(144), plt.imshow(blur3), plt.title("Median Filtering")
plt.xticks([]), plt.yticks([])
plt.show()
```

5.1 Image Processing

◆ Import Module

Firstly, import the required module through import statement.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

◆ Read Images

Then call **imread()** function in cv2 module to read the image that needs to be filtered.

```
img = cv2.imread('noise.jpg')
```

In the bracket is the image name.

◆ **Mean Filtering**

Call blur() function in cv2 module to perform mean filtering on the specific image.

```
9   blur1 = cv2.blur(img, (5, 5))    # mean filtering
```

The format of blur() function is as follow.

```
cv2.blur(scr, ksize)
```

The first parameter "**scr**" is the input image.

The second parameter "**ksize**" is the size of convolution kernel.

◆ **Gauss Filtering**

Call GaussianBlur() function in cv2 module to perform Gauss filtering on the specific image.

```
10   blur2 = cv2.GaussianBlur(img, (5, 5), 1)    # Gauss filtering
```

The format of GaussianBlur() function is as follow.

```
cv2.GaussianBlur(src, ksize, sigmaX, sigmaY, borderType)
```

The first parameter "**scr**" is the input image.

The second parameter "**ksize**" is the size of Gaussian convolution kernel. Both the height and width of the convolution kernel must be positive number and odd number.

The third parameter "**sigmaX**" is the horizontal standard deviation of Gaussian kernel.

The fourth parameter "**sigmaY**" is the vertical standard deviation of

Gaussian kernel, **0** by default.

The fifth parameter “**borderType**” is the type of border filling.

◆ **Median Filtering**

Call medianBlur() function in cv2 module to perform median filtering on the specific image.

```
11    blur3 = cv2.medianBlur(img, 5)   # Median filtering
```

The format of medianBlur() function is as follow

```
cv2.medianBlur(src, ksize)
```

The first parameter “**scr**” is the input image.

The second parameter “**ksize**” is the size of the convolution kernel.

5.2 Image Display

◆ **Create Custom Image**

Call figure() function in matplotlib.pyplot module to create a custom image for displaying the final output image.

```
14    plt.figure(figsize=(10, 5), dpi=100)
```

The format of the figure() function is as follow.

```
matplotlib.pyplot.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True, FigureClass=<class 'matplotlib.figure.Figure'>, clear=False, **kwargs)
```

The first parameter “**num**” is the only identifier of the image i.e. the serial number of the picture (number) or the name (string).

The second parameter “**figsize**” is the width and height of the image in inch.

The third parameter “**dpi**” is the resolution of the image i.e. the number of pixels by inch

The fourth parameter “**facecolor**” is the background color.

The fifth parameter “**edgecolor**” is the frame color

The sixth parameter “**frameon**” determines whether to draw the picture, and it is “**True**” by default.

The seventh parameter “**FigureClass**” is used to select the custom figure when generating the image

The eighth parameter “**clear**” determines whether to clear all the original images.

The ninth parameter “****kwargs**” represents other properties of the image.

- **Modify matplotlib Configuration**

matplotlib is plotting library of Python. User can access and modify matplotlib configuration options through parameter dictionary “**rcParams**”.

```
plt.rcParams['axes.unicode_minus'] = False
```

The codes above are used to manipulate the display of the normal characters.

- **Set the Parameter of Image Display**

Call subplot(), imshow() and title() functions in matplotlib.pyplot modules to designate the position, color and headline of the subplot in the Figure.

```
plt.subplot(141), plt.imshow(img), plt.title("Original")
```

1) subplot() function is used to set the position of the subplot, and the

function format is as follow.

```
matplotlib.pyplot.subplot(nrows, ncols, index, **kwargs)
```

The first parameter "**nrows**" and the second parameter "**ncols**" respectively are the number of row and column of subplot.

The third parameter "**index**" is the index position. Index starts at 1 in the upper left corner and increases to the right.

When both the row and column are less than "**10**", these two values can be abbreviated to an integer. For example, the meaning of "subplot(1, 4, 1)" and "subplot(141)" are the same, both representing the image is divided into one row and four columns, and the subplot is in the first place i.e. 1st row, 1st column.

2） imshow() function is used to set the color of subplot, and its format is as follow.

```
matplotlib.pyplot.imshow(X, cmap=None)
```

The first parameter "**X**" is the image data.

The second parameter "**cmap**" is the colormap, RGB(A) color space by default.

3） title() function is used to set the title of the subplot. The parameter in the bracket is the name of the subplot and the function format is as follow.

```
matplotlib.pyplot.title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs)
```

The first parameter "**label**" is the title composed of string.

The second parameter "**fontdict**" is the property of the font, and the current parameter refers to dictionary.

The third parameter “**loc**” is the position of the title. It can be “**left**”, “**center**” or “**right**”, and “**center**” by default.

The fourth parameter “**pad**” is the padding distance (inside margin) between the tile and the subplot, “**6.0**” by default.

The fifth parameter “**y**” is the vertical distance between the title and the subplot, and the unit is the percentage of the height of the subplot. The default value is "None", that is, the position of the title is automatically determined to avoid overlapping with other elements. "**1.0**" means the title is at the top of the subplot.

The sixth parameter "****kwargs**" is the text object keyword property, which is used to determine the appearance of the text, such as font, text color, etc.

◆ **Set Axis Tick**

Call xticks() and yticks() function in matplotlib.pyplot module to set the tick and tag of X and Y axis. As the coordinate axis is not required in image display in this routine, the list is set as none that is the coordinate axis will not be displayed.

```
plt.xticks([]), plt.yticks([])
```

The format of xticks() function is as follow.

```
xticks(ticks=None, labels=None, **kwargs)
```

When the parameter is none, the function will return the current tick and tag of X axis. Otherwise the function is used to set the current tick and label of X axis.

The first parameter "**ticks**" is a list of the positions of the X-axis ticks. If the list is empty, the X-axis ticks will be cleared.

The second parameter “**labels**” is the label of X-axis tick. Only when parameter “ticks” is not none, can this parameter be passed.

The third parameter “****kwargs**” is used to control the appearance of the tick and label.

The format of yticks() is the same as that of xticks(). The difference lies in the controlled object.

◆ **Display Image**

Call show() function in matplotlib.pyplot module to display the image on the window.

```
plt.show()
```

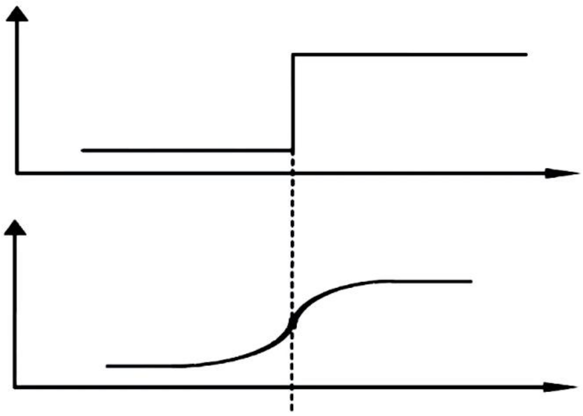
The complete codes of image display part are as follow.

```
plt.figure(figsize=(10, 5), dpi=100)
plt.rcParams['axes.unicode_minus'] = False
plt.subplot(141), plt.imshow(img), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(142), plt.imshow(blur1), plt.title("Mean Filtering")
plt.xticks([]), plt.yticks([])
plt.subplot(143), plt.imshow(blur2), plt.title("Gauss Filtering")
plt.xticks([]), plt.yticks([])
plt.subplot(144), plt.imshow(blur3), plt.title("Median Filtering")
plt.xticks([]), plt.yticks([])
plt.show()
```

Lesson 10 Image Processing---Edge Detection

1.Edge Detection Introduction

Edge detection is fundamental technique in image processing and computer vision, which aims at identifying edges in a digital image at which the image brightness changes sharply. Sharp changes in image usually reflect important events and changes in properties. The edge is as the picture shown.



Edge detection greatly reduces the amount of data, removes irrelevant information, and preserves the important structural properties of the image. Edge detection is divided into two types.

1）Based on search: The boundary is detected by finding the maximum and minimum values in the first derivative of the image, and it usually locates in the direction with the largest gradient. The representative algorithms are the Sobel operator and the Scharr operator.

2）Based on zero-crossing: the boundary is found by searching the second-order derivative zero-crossing of the image, which is usually the Laplacian zero-crossing point or the zero-crossing point represented by the nonlinear difference, and the representative algorithm is the Laplacian operator.

2.Canny Edge Detection

Canny Edge Detection is a popular edge detection algorithm. It was developed by John F. Canny in 1986 and considered as the best algorithm of edge detection. Canny edge detection will go through 4 steps, including Noise Reduction, Finding Gradient Magnitude and Direction of the Image, Non-maximum Suppression and Hysteresis Thresholding.

- **Noise Reduction**

Since edge detection is susceptible to noise in the image, first step is to remove the noise in the image. For detailed operation, please refer to the file in "**4. OpenCV Computer Vision Lesson->Lesson 3 Image Processing---Smoothing**".

- **Finding Gradient Magnitude and Direction of the Image**

In math, gradient is a vector, indicating that the directional derivative of the function at a certain point reaches the maximum along this direction, that is, the function changes the fastest along the direction at this point, and the rate of change is the greatest.

In the image, gradient represents the degree and direction of gray value change, and the edge refers to the position where the gray intensity changes the most. Gradient direction is always perpendicular to edges.

Sobel filter is used to calculate the magnitude and direction of the gradient. The Sobel operator G_X is the first derivative in the horizontal direction, which is used to detect the edge in the Y-axis direction, while G_Y is the the first derivative in the vertical direction, which is used to detect the edge in the X-axis direction.

The formula for calculating the gradient size is as follows

$$G = \sqrt{(G_X^2 + G_Y^2)}$$

The formula for calculating the gradient direction is as follows

$$\theta = arctan\frac{G_Y}{G_X}$$

- **Non-Maximum Suppression**

Non-Maximum Suppression (NMS) is that reserve local maximum and suppress all the values apart from local maximum. In simple terms, all the pixels of the image will be detected. If the gradient intensity of a point is greater than the pixels in the positive and negative directions of its gradient direction, the point is retained; otherwise, the point is suppressed.

Canny edge detection algorithm perform non-maximum suppression along the gradient direction not the edge direction.

- Hysteresis Thresholding

This stage decides which are really edges. For this, we need two threshold values, “**minVal**” and “**maxVal**”. Any edges with intensity gradient more than maxVal are sure to be edges and those below minVal are sure to be non-edges. Those who lie between these two thresholds are classified edges or non-edges based on their connectivity. If they are connected to "sure-edge" pixels, they are considered to be part of edges. Otherwise, they are also discarded.

3.Operation Steps

This routine will perform the edge detection.

Before operation, please copy the routine “**edge_detection.py**”and sample picture “**luna.jpg**” in “4.OpenCV->Lesson 8 Image Processing --- Edge

Detection->Routine Code" to the shared folder.

For how to configure the shared folder, please refer to the file in "**2. Linux Basic Lesson->Lesson 3 Linux Installation and Source Replacemen**t".

Note: the input command should be case sensitive and the keywords can be complemented by "Tab" key.

1） Open virtual machine and start the system. Click "", and then "" or press "**Ctrl+Alt+T**" to open command line terminal.

2） Input command "**cd /mnt/hgfs/Share/**" and press Enter to enter the shared folder.

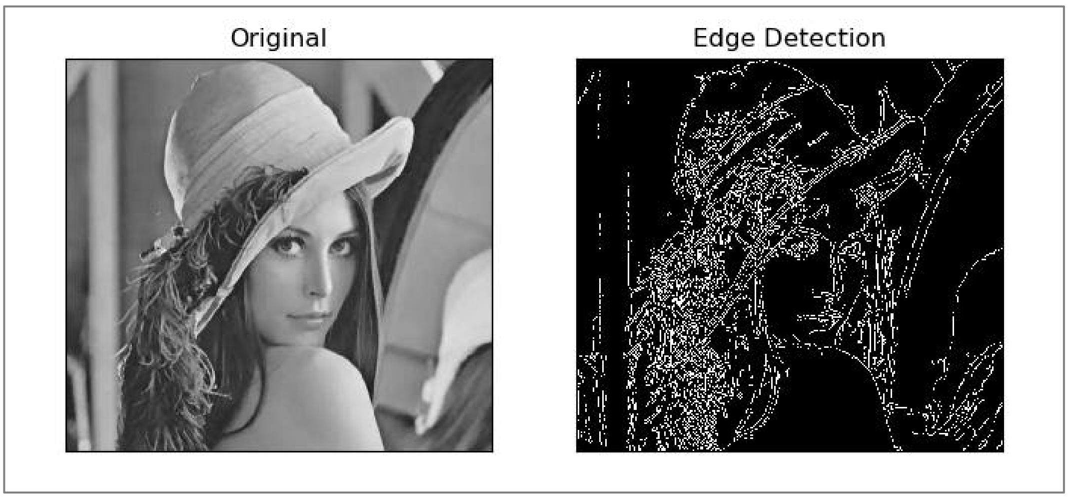
```
hiwonder@ubuntu:~$ cd /mnt/hgfs/Share/
```

3） Input command "**python3 edge_detection.py**" and press Enter to run the routine.

```
hiwonder@ubuntu:/mnt/hgfs/Share$ python3 edge_detection.py
```

4.Program Outcome

The final output image is as follow.



5.Program Analysis

The routine “**edge_detection.py**” can be found in “**4. OpenCV Computer Vision Lesson->Lesson 8 Image Processing---Edge Detection->Routine Code**”.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# read the image
img = cv2.imread('luna.jpg')

# Canny edge detection
lowThreshold = 1
max_lowThreshold = 80
canny = cv2.Canny(img, lowThreshold, max_lowThreshold)

# image display
plt.figure(figsize=(8, 5), dpi=100)
plt.rcParams['axes.unicode_minus'] = False
plt.subplot(121), plt.imshow(img, cmap=plt.cm.gray), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(canny, cmap=plt.cm.gray), plt.title("Edge Detection")
plt.xticks([]), plt.yticks([])
plt.show()
```

5.1 Image Processing

◆ Import Module

Firstly, import the required module through import statement.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

◆ Read Image

Call **imread()** function in cv2 module to read the image

```
img = cv2.imread('luna.jpg')
```

The parameter in the bracket is the name of the image.

◆ Set Threshold

Set two thresholds to determine the final edge.

```
lowThreshold = 1
max_lowThreshold = 80
```

◆ **Edge Detection**

Call Canny() function in cv2 module to perform edge detection on the specific image.

```
canny = cv2.Canny(img, lowThreshold, max_lowThreshold)
```

The format of Canny() function is as follow.

```
Canny( image, threshold1, threshold2)
```

The first parameter "**image**" is the input image.

The second parameter "**threshold1**" is the low threshold "**minVal**"

The third parameter "**threshold2**" is high threshold "**maxVal**".

5.2 Image Display

◆ **Create Custom Figure**

Call figure() function in matplotlib.pyplot module to create a custom figure for displaying the final output image.

```
plt.figure(figsize=(8, 5), dpi=100)
```

The format of the figure() function is as follow.

```
figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True, FigureClass=<class 'matplotlib.figure.Figure'>, clear=False, **kwargs)
```

The first parameter "**num**" is the only identifier of the image i.e. the serial number of the picture (number) or the name (string).

The second parameter “**figsize**” is the width and height of the image in inch.

The third parameter “**dpi**” is the resolution of the image i.e. the number of pixels by inch

The fourth parameter “**facecolor**” is the background color.

The fifth parameter “**edgecolor**” is the frame color

The sixth parameter “**frameon**” determines whether to draw the picture, and it is “**True**” by default.

The seventh parameter “**FigureClass**” is used to select the custom figure when generating the image

The eighth parameter “**clear**” determines whether to clear all the original images.

The ninth parameter “****kwargs**” represents other properties of the image.

- **Modify matplotlib Configuration**

matplotlib is plotting library of Python. User can access and modify matplotlib configuration options through parameter dictionary “**rcParams**”.

```
plt.rcParams['axes.unicode_minus'] = False
```

The codes above are used to manipulate the display of the normal characters.

- **Set the Parameter of Image Display**

Call subplot(), imshow() and title() functions in matplotlib.pyplot modules to designate the position, color and headline of the subplot in the Figure.

```
plt.subplot(121), plt.imshow(img, cmap=plt.cm.gray), plt.title("Original")
```

1） subplot() function is used to set the position of the subplot, and the function format is as follow.

```
subplot(nrows, ncols, index, **kwargs)
```

The first parameter “**nrows**” and the second parameter “**ncols**” respectively are the number of row and column of subplot.

The third parameter “**index**” is the index position. Index starts at 1 in the upper left corner and increases to the right.

When both the row and column are less than “**10**”, these two values can be abbreviated to an integer. For example, the meaning of “subplot(1, 2, 1)” and “subplot(121)” are the same, both representing the image is divided into one row and 2 columns, and the subplot is in the first place i.e. 1st row, 1st column.

2） imshow() function is used to set the color of subplot, and its format is as follow.

```
imshow(X, cmap=None)
```

The first parameter “**X**” is the image data.

The second parameter “**cmap**” is the colormap, RGB(A) color space by default.

3） title() function is used to set the title of the subplot. The parameter in the bracket is the name of the subplot and the function format is as follow.

```
title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs)
```

The first parameter “label” is the title composed of string.

The second parameter “fontdict” is the property of the font, and the current parameter refers to dictionary.

The third parameter “loc” is the position of the title. It can be “left”, “center” or “right”, and “center” by default.

The fourth parameter “pad” is the padding distance (inside margin) between the tile and the subplot, “6.0” by default.

The fifth parameter “y” is the vertical distance between the title and the subplot, and the unit is the percentage of the height of the subplot. The default value is "None", that is, the position of the title is automatically determined to avoid overlapping with other elements. "1.0" means the title is at the top of the subplot.

The sixth parameter "**kwargs" is the text object keyword property, which is used to determine the appearance of the text, such as font, text color, etc.

- **Set Axis Tick**

Call xticks() and yticks() function in matplotlib.pyplot module to set the tick and tag of X and Y axis. As the coordinate axis is not required in image display in this routine, the list is set as none that is the coordinate axis will not be displayed.

```
plt.xticks([]), plt.yticks([])
```

The format of xticks() function is as follow.

```
xticks(ticks=None, labels=None, **kwargs)
```

When the parameter is none, the function will return the current tick and tag of X axis. Otherwise the function is used to set the current tick and label of X axis.

The first parameter "**ticks**" is a list of the positions of the X-axis ticks. If the list is empty, the X-axis ticks will be cleared.

The second parameter “**labels**” is the label of X-axis tick. Only when parameter “ticks” is not none, can this parameter be passed.

The third parameter “****kwargs**” is used to control the appearance of the tick and label.

The format of yticks() is the same as that of xticks(). The difference lies in the controlled object.

◆ **Display Image**

Call show() function in matplotlib.pyplot module to display the image on the window.

```
plt.show()
```

The complete codes of image display part are as follow.

```
plt.figure(figsize=(8, 5), dpi=100)
plt.rcParams['axes.unicode_minus'] = False
plt.subplot(121), plt.imshow(img, cmap=plt.cm.gray), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(canny, cmap=plt.cm.gray), plt.title("Edge Detection")
plt.xticks([]), plt.yticks([])
plt.show()
```

Lesson 11 Image Processing --- Morphological Processing

1.Morphology Introduction

Morphology is one of the most widely used techniques in image processing. It is mainly used to extract image components that are meaningful for describing the shape of an area, so that the most essential shape features of the target object can captured in subsequent recognition, such as boundaries and connected areas. In addition, techniques such as thinning, pixelation, and burr trimming are often used in image preprocessing and postprocessing, which can greatly strengthen the image.

The basic idea of morphology is to use a special structural element to measure or extract the corresponding shape or feature in the input image for further image analysis and target recognition.

2.Morphological Transformation

2.1 Erosion and Dilation

Both erosion and dilation are the basic and important morphological operation, and also the foundations of multiple advanced morphological processing. Many morphological algorithms are composed of these two.

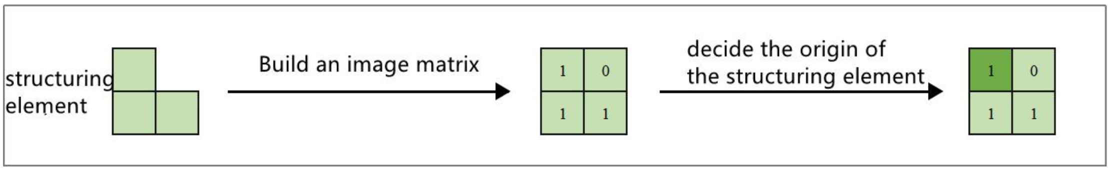
- **Structuring Element**

Structuring element is required in erosion and dilation. A two-dimensional structuring element can be seen as a two-dimensional matrix element which is "**0**" or "**1**".

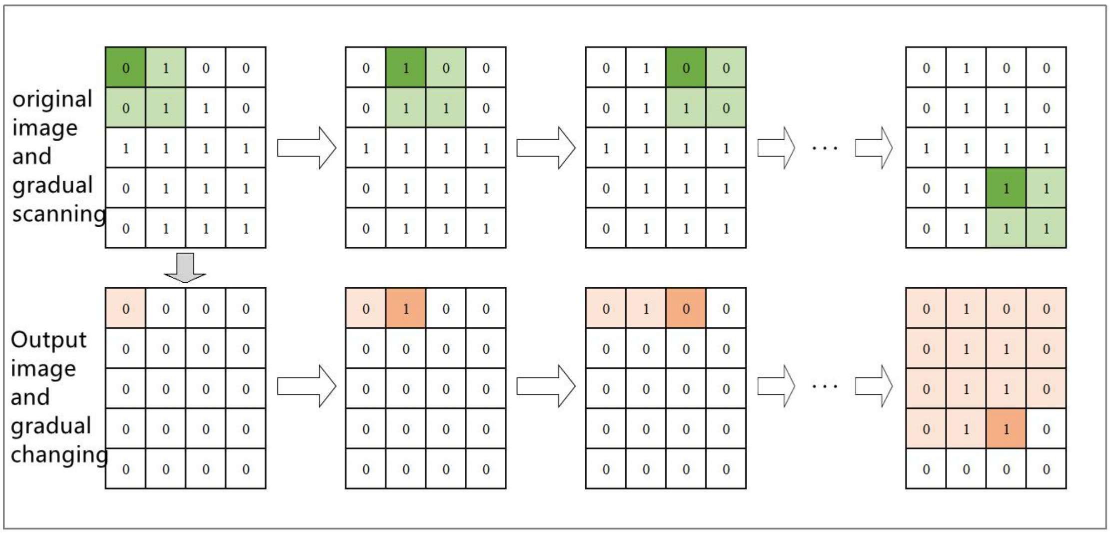
◆ Erosion

Erosion works to remove small and meaningless object, and the whole process is divided into three steps.

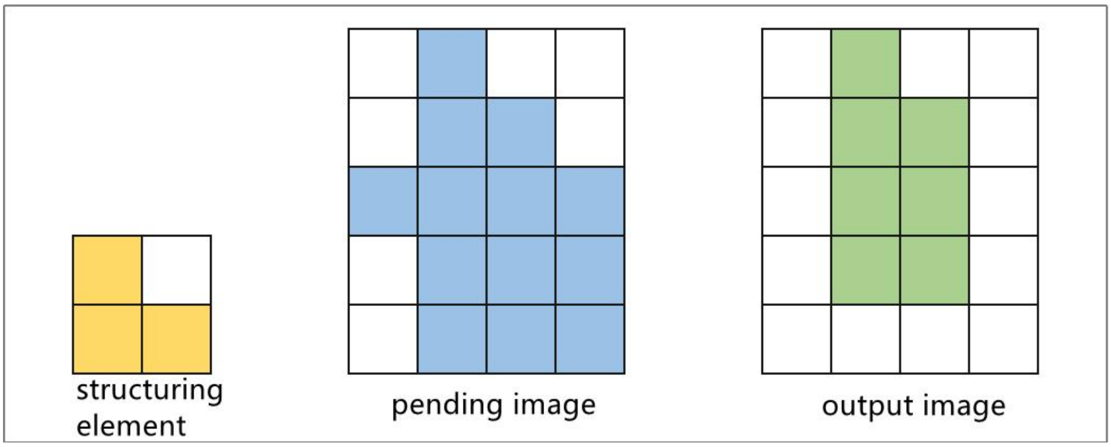
1）Build an image matrix upon the structuring element and determine its origin. Take the element at the upper left corner as the origin, and mark it with dark color.



2）Overlay the structuring element on the pending image. If the value of the pixel in the image corresponding to the elements whose value is “**1**” in the structuring elements are all “1”, the pixel at the corresponding position of the origin is assigned as "**1**", otherwise it is "**0**".



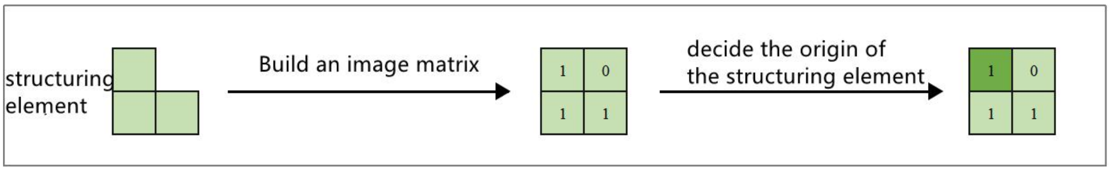
3）Make the structuring elements move on the pending image in order until all the images are processed completely.



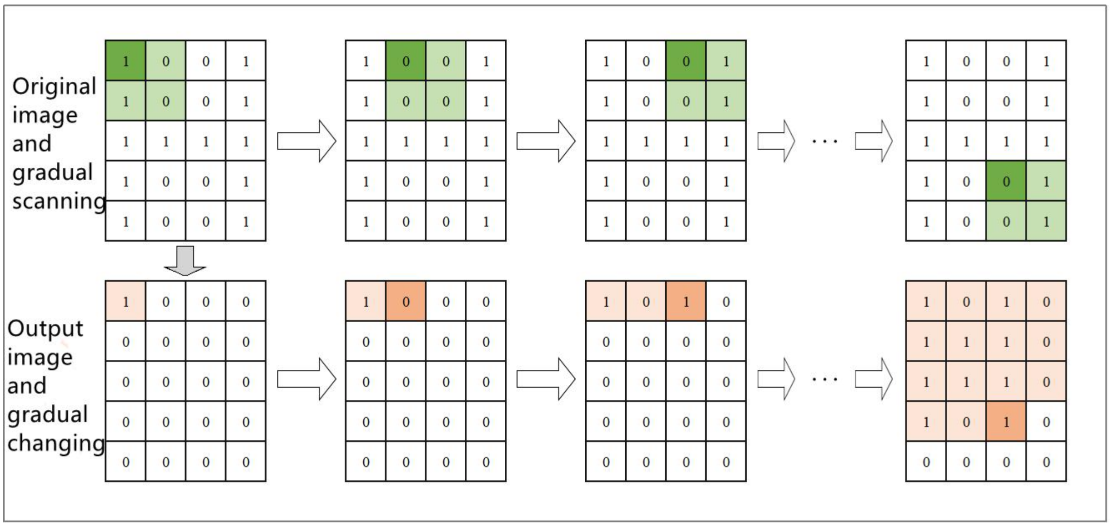
◆ **Dilation**

Dilation can enlarge the edge of the image and pad the edge of the target object or non-target pixel. The operations are divided into three steps.

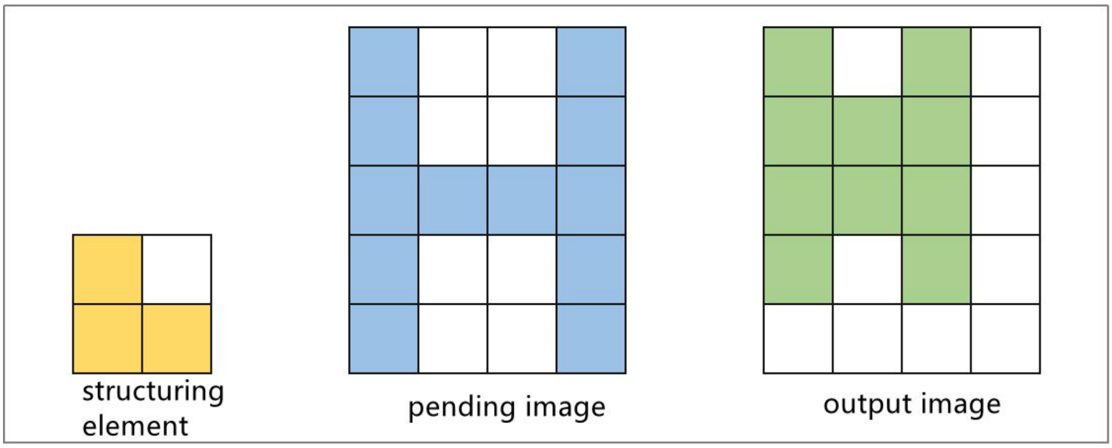
1） Build an image matrix upon the structuring element and determine its origin. Take the element at the upper left corner as the origin, and mark it with dark color.



2） Overlay the structuring element on the pending image. If at least one of the value of the pixel in the image corresponding to the elements whose value is “**1**” in the structuring elements is “1”, the pixel at the corresponding position of the origin is assigned as "**1**", otherwise it is "**0**".



3） Make the structuring elements move on the pending image in order until all the images are processed completely.



2.2 Opening and Closing

In opening and closing, erosion and dilation are executed in sequence.

- **Open Operation**

Opening indicates that erosion is executed first and dilation follows. It is useful in separating objects, removing small area, removing highlight under dark background.

- **Close Operation**

In closing, dilation is executed first and erosion follows. It plays an important role in eliminating holes, that is, filling closed areas and deleting dark areas under a bright background.

2.3 Top Hat and Bottom Hat

◆ **Top Hat Operation**

It is the difference between input image and the image after opening (Top hat operation= input image - image after opening), and it can obtain areas with brighter gray in the original image

◆ **Bottom Hat Operation (Black Hat)**

It is the difference between input image and the image after closing (Top hat operation= input image - image after closing), and it can obtain areas with darker gray in the original image

3.Operation Steps

This routine will perform erosion, dilation, opening, closing, top hat operation and bottom hat operation on the designated image.

Before operation, please copy the routine "**example_org.jpg**" in "4.OpenCV->Lesson 11 Image Processing --- Geometric Transformation->Routine Code" to the shared folder.

For how to configure the shared folder, please refer to the file in "**2. Linux Basic Lesson->Lesson 3 Linux Installation and Source Replacement**".

Note: the input command should be case sensitive and the keywords can be complemented by "Tab" key.

1） Open virtual machine and start the system. Click "", and then "" or press “**Ctrl+Alt+T**” to open command line terminal.

2） Input command “**cd /mnt/hgfs/Share/**” and press Enter to enter the shared folder.

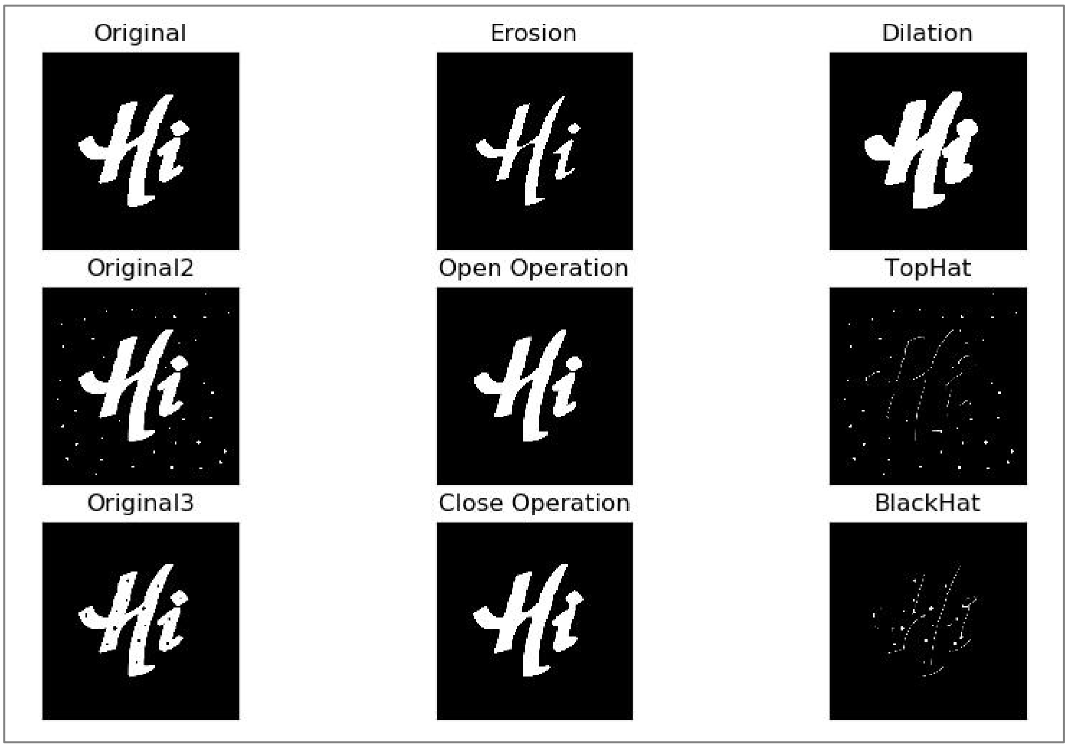
```
hiwonder@ubuntu:~$ cd /mnt/hgfs/Share/
```

3） Input command “**python3 morphology operations.py**” and press Enter to run the routine.

```
hiwonder@ubuntu:/mnt/hgfs/Share$ python3 morphology_operations.py
```

4.Program Outcome

The final output image is as follow.



5.Program Analysis

The routine “**morphology_operations.py**” can be found in “4. OpenCV Computer Vision Lesson->Lesson 11 Image Processing---Morphological Processing->Routine Code”.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# read the image
img_org = cv2.imread('example_org.jpg')
img_noise = cv2.imread('example_noise.jpg')
img_cave = cv2.imread('example_cave.jpg')

# build nuclear structure
kernel = np.ones((10, 10), np.uint8)  # 10*10 all-one matrix

# Morphological processing
erosion_img = cv2.erode(img_org, kernel)  # erosion
dilate_img = cv2.dilate(img_org, kernel)  # dilation
open_img = cv2.morphologyEx(img_noise, cv2.MORPH_OPEN, kernel)  # open operation
close_img = cv2.morphologyEx(img_cave, cv2.MORPH_CLOSE, kernel)  # close operation
top_hat_img = cv2.morphologyEx(img_noise, cv2.MORPH_TOPHAT, kernel)  # top hat operation
black_hat_img = cv2.morphologyEx(img_cave, cv2.MORPH_BLACKHAT, kernel)  # bottom hat operation

# image display
plt.figure(figsize=(10, 6), dpi=100)
plt.rcParams['axes.unicode_minus'] = False

plt.subplot(331), plt.imshow(img_org), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(332), plt.imshow(erosion_img), plt.title("Erosion")
plt.xticks([]), plt.yticks([])
plt.subplot(333), plt.imshow(dilate_img), plt.title("Dilation")
plt.xticks([]), plt.yticks([])

plt.subplot(334), plt.imshow(img_noise), plt.title("Original2")
plt.xticks([]), plt.yticks([])
plt.subplot(335), plt.imshow(open_img), plt.title("Open Operation")
plt.xticks([]), plt.yticks([])
plt.subplot(336), plt.imshow(top_hat_img), plt.title("TopHat")
plt.xticks([]), plt.yticks([])

plt.subplot(337), plt.imshow(img_cave), plt.title("Original3")
plt.xticks([]), plt.yticks([])
plt.subplot(338), plt.imshow(close_img), plt.title("Close Operation")
plt.xticks([]), plt.yticks([])
plt.subplot(339), plt.imshow(black_hat_img), plt.title("BlackHat")
plt.xticks([]), plt.yticks([])

plt.show()
```

5.1 Image Processing

- **Import Module**

Firstly, import the required module through import statement.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

◆ **Read the Image**

Then call **imread()** function in cv2 module to read the pending image.

```
img_org = cv2.imread('example_org.jpg')
img_noise = cv2.imread('example_noise.jpg')
img_cave = cv2.imread('example_cave.jpg')
```

The parameter in the bracket is the name of the image

◆ **Create Nuclear Structure**

Call ones() function in numpy module to create the nuclear structure i.e. array required in the operation.

```
kernel = np.ones((10, 10), np.uint8)  # 10*10 all-one matrix
```

The format of ones() function is as follow.

```
np.ones(shape, dtype=None, order='C')
```

The first parameter "**shape**" is a integer or integer tuple used to define the size of the array. If it designates the variable of the integer, one--dimensional array will be returned. If it designate integer tuple, the array in given shape will be returned.

The second parameter "**dtype**" refers to the data type of the array, "**float**" by default.

The third parameter "**order**" is used to designate the storing order of the returned array elements in storage.

◆ **Erosion and Dilation**

Call erode() and dilate() function in cv2 module to perform erosion and dilation on the specific image.

```
erosion_img = cv2.erode(img_org, kernel)  # erosion
dilate_img = cv2.dilate(img_org, kernel)  # dilation
```

The format of erode() function is as follow.

cv2.erode(src, kernel, iteration)

The first parameter "**src**" is the input image.

The second parameter "**kernel**" is the size of the kernel.

The third parameter is the umber of iteration.

The meaning of the parameter in dilate() function is the same as that of erode() function.

◆ **Open Operation and Close Operation**

Call morphologyEx() function in cv2 module to perform open operation, close operation, top hat operation and bottom hat operation on the specific image.

```
open_img = cv2.morphologyEx(img_noise, cv2.MORPH_OPEN, kernel)  # open operation
close_img = cv2.morphologyEx(img_cave, cv2.MORPH_CLOSE, kernel)  # close operation
top_hat_img = cv2.morphologyEx(img_noise, cv2.MORPH_TOPHAT, kernel)  # top hat operation
black_hat_img = cv2.morphologyEx(img_cave, cv2.MORPH_BLACKHAT, kernel)  # bottom hat operation
```

The format of morphologyEx() function is as follow.

cv2.morphologyEx(img, op, kernel)

The first parameter "**img**" indicates the input image

The second parameter "**op**"represents the operation type.

Operation Type	Meaning
cv2.MORPH_OPEN	open operation
cv2.MORPH_CLOSE	close operation

cv2.MORPH_GRADIENT	morphological gradient
cv2.MORPH_TOPHAT	top hat operation
cv2.MORPH_BLACKHAT	bottom hat operation

The third parameter indicates the size of the frame.

5.2 Image Display

◆ **Create Custom Figure**

Call figure() function in matplotlib.pyplot module to create a custom figure for displaying the final output image.

```
plt.figure(figsize=(10, 6), dpi=100)
```

The format of the figure() function is as follow.

matplotlib.pyplot.figure(num=None, figsize=None, dpi=None, facecolor=None, edgecolor=None, frameon=True, FigureClass=<class 'matplotlib.figure.Figure'>, clear=False, **kwargs)

The first parameter "**num**" is the only identifier of the image i.e. the serial number of the picture (number) or the name (string).

The second parameter "**figsize**" is the width and height of the image in inch.

The third parameter "**dpi**" is the resolution of the image i.e. the number of pixels by inch

The fourth parameter "**facecolor**" is the background color.

The fifth parameter "**edgecolor**" is the frame color

The sixth parameter “**frameon**” determines whether to draw the picture, and it is “**True**” by default.

The seventh parameter “**FigureClass**” is used to select the custom figure when generating the image

The eighth parameter “**clear**” determines whether to clear all the original images.

The ninth parameter “****kwargs**” represents other properties of the image.

◆ Modify matplotlib Configuration

matplotlib is plotting library of Python. User can access and modify matplotlib configuration options through parameter dictionary “**rcParams**”.

```
plt.rcParams['axes.unicode_minus'] = False
```

The codes above are used to manipulate the display of the normal characters.

◆ **Set the Parameter of Image Display**

Call subplot(), imshow() and title() functions in matplotlib.pyplot modules to designate the position, color and headline of the subplot in the Figure.

```
plt.subplot(331), plt.imshow(img_org), plt.title("Original")
```

1） subplot() function is used to set the position of the subplot, and the function format is as follow.

```
matplotlib.pyplot.subplot(nrows, ncols, index, **kwargs)
```

The first parameter “**nrows**” and the second parameter “**ncols**” respectively are the number of row and column of subplot.

The third parameter “**index**” is the index position. Index starts at 1 in the upper left corner and increases to the right.

When both the row and column are less than “**10**”, these two values can be abbreviated to an integer. For example, the meaning of “**subplot(3, 3, 1)**” and “**subplot(331)**” are the same, both representing the image is divided into three rows and three columns, and the subplot is in the first place i.e. 1^{st} row, 1^{st} column.

2） imshow() function is used to set the color of subplot, and its format is as follow.

```
matplotlib.pyplot.imshow(X, cmap=None)
```

The first parameter “**X**” is the image data.

The second parameter “**cmap**” is the colormap, RGB(A) color space by default.

3） title() function is used to set the title of the subplot. The parameter in the bracket is the name of the subplot and the function format is as follow.

```
matplotlib.pyplot.title(label, fontdict=None, loc=None, pad=None, *, y=None, **kwargs)
```

The first parameter “label” is the title composed of string.

The second parameter “fontdict” is the property of the font, and the current parameter refers to dictionary.

The third parameter “loc” is the position of the title. It can be “left”, “center” or “right”, and “center” by default.

The fourth parameter “pad” is the padding distance (inside margin) between the tile and the subplot, “6.0” by default.

The fifth parameter “y” is the vertical distance between the title and the subplot, and the unit is the percentage of the height of the subplot. The default value is "None", that is, the position of the title is automatically determined to avoid overlapping with other elements. "1.0" means the title is at the top of the subplot.

The sixth parameter "**kwargs" is the text object keyword property, which is used to determine the appearance of the text, such as font, text color, etc.

- **Set Axis Tick**

Call xticks() and yticks() function in matplotlib.pyplot module to set the tick and tag of X and Y axis. As the coordinate axis is not required in image display in this routine, the list is set as none that is the coordinate axis will not be displayed.

```
26    plt.xticks([]), plt.yticks([])
```

The format of xticks() function is as follow.

```
matplotlib.pyplot.xticks(ticks=None, labels=None, **kwargs)
```

The first parameter "**ticks**" is a list of the positions of the X-axis ticks. If the list is empty, the X-axis ticks will be cleared.

The second parameter “**labels**” is the label of X-axis tick. Only when parameter “ticks” is not none, can this parameter be passed.

The third parameter “****kwargs**” is used to control the appearance of the tick and label.

The format of yticks() is the same as that of xticks(). The difference lies in the controlled object.

- **Display Image**

Call show() function in matplotlib.pyplot module to display the image on the window.

```
46    plt.show()
```

The complete codes of image display part are as follow.

```
plt.subplot(331), plt.imshow(img_org), plt.title("Original")
plt.xticks([]), plt.yticks([])
plt.subplot(332), plt.imshow(erosion_img), plt.title("Erosion")
plt.xticks([]), plt.yticks([])
plt.subplot(333), plt.imshow(dilate_img), plt.title("Dilation")
plt.xticks([]), plt.yticks([])

plt.subplot(334), plt.imshow(img_noise), plt.title("Original2")
plt.xticks([]), plt.yticks([])
plt.subplot(335), plt.imshow(open_img), plt.title("Open Operation")
plt.xticks([]), plt.yticks([])
plt.subplot(336), plt.imshow(top_hat_img), plt.title("TopHat")
plt.xticks([]), plt.yticks([])

plt.subplot(337), plt.imshow(img_cave), plt.title("Original3")
plt.xticks([]), plt.yticks([])
plt.subplot(338), plt.imshow(close_img), plt.title("Close Operation")
plt.xticks([]), plt.yticks([])
plt.subplot(339), plt.imshow(black_hat_img), plt.title("BlackHat")
plt.xticks([]), plt.yticks([])

plt.show()
```